

```

# =====
# SISTEMA DE ANÁLISE DE TRELIÇAS PLANAS ISOSTÁTICAS
# Implementação usando vetores e eliminação de Gauss manual
# =====

import math
import time
import numpy as np
import matplotlib.pyplot as plt

# =====
# FUNÇÃO: ELIMINAÇÃO DE GAUSS MANUAL
# =====

def gauss(A, b):

    n = len(b)

    # Matriz aumentada
    M = []

    for i in range(n):
        linha = A[i][:] + [b[i]]
        M.append(linha)

    # Eliminação
    for k in range(n):

        # Pivotamento parcial
        maior = abs(M[k][k])
        linha_pivo = k

        for i in range(k + 1, n):
            if abs(M[i][k]) > maior:
                maior = abs(M[i][k])
                linha_pivo = i

        M[k], M[linha_pivo] = M[linha_pivo], M[k]

    # Verificação de singularidade
    if abs(M[k][k]) < 1e-12:
        raise ValueError("Sistema singular ou mecanismo
estrutural.")

```

```

# Eliminação
for i in range(k + 1, n):

    fator = M[i][k] / M[k][k]

    for j in range(k, n + 1):
        M[i][j] -= fator * M[k][j]

# Retrossubstituição
x = [0.0] * n

for i in range(n - 1, -1, -1):

    soma = 0.0

    for j in range(i + 1, n):
        soma += M[i][j] * x[j]

    x[i] = (M[i][n] - soma) / M[i][i]

return x

# =====
# FUNÇÃO: MONTAGEM DO SISTEMA DE EQUILÍBRIO
# =====

def montar_sistema(N, B, x, y, barras, Fx, Fy, apoios):

    # -----
    # REAÇÕES DE APOIO
    # fixo -> Rx e Ry
    # movel -> Ry
    # -----

    reacoes = []

    for i in range(N):

        if apoios[i] == "fixo":
            reacoes.append((i, 'x'))
            reacoes.append((i, 'y'))

```

```

        elif apoios[i] == "movel":
            reacoes.append((i, 'y'))

R = len(reacoes)

# Número de incógnitas
total_incognitas = B + R

# Verificação de isostaticidade
if total_incognitas != 2 * N:
    print("\nATENÇÃO:")
    print("A estrutura pode não ser isostática.")
    print(f"B + R = {total_incognitas}")
    print(f"2N      = {2*N}\n")

# -----
# MATRIZ DE EQUILÍBRIO
# -----

A = [[0.0 for _ in range(total_incognitas)] for _ in range(2 * N)]

b = [0.0 for _ in range(2 * N)]

# =====
# BARRAS
# =====

for k in range(B):

    i = barras[k][0]
    j = barras[k][1]

    dx = x[j] - x[i]
    dy = y[j] - y[i]

    L = math.sqrt(dx**2 + dy**2)

    c = dx / L
    s = dy / L

    # Equação em x
    A[2*i][k] = c

```

```

    A[2*j][k] = -c

    # Equação em y
    A[2*i + 1][k] = s
    A[2*j + 1][k] = -s

# =====
# REAÇÕES
# =====

for r in range(R):

    no, direcao = reacoes[r]

    coluna = B + r

    if direcao == 'x':
        A[2*no][coluna] = 1.0

    elif direcao == 'y':
        A[2*no + 1][coluna] = 1.0

# =====
# VETOR DE CARGAS
# =====

for i in range(N):

    b[2*i] = -Fx[i]
    b[2*i + 1] = -Fy[i]

    return A, b, reacoes

# =====
# FUNÇÃO: CLASSIFICAÇÃO DAS BARRAS
# =====

def classificar_barras(forcas):

    tipos = []

    for f in forcas:

```

```

        if abs(f) < 1e-6:
            tipos.append("NULA")

        elif f > 0:
            tipos.append("TRAÇÃO")

        else:
            tipos.append("COMPRESSÃO")

    return tipos

# =====
# FUNÇÃO: ESTATÍSTICAS
# =====

def estatisticas(forcas, reacoes_valores):

    abs_forcas = [abs(f) for f in forcas]

    maior = max(abs_forcas)
    indice = abs_forcas.index(maior)

    media = sum(abs_forcas) / len(abs_forcas)

    soma_reacoes = sum(abs(r) for r in reacoes_valores)

    print("\n===== ESTATÍSTICAS =====")
    print(f"Barra mais solicitada: Barra {indice}")
    print(f"Força máxima: {maior:.2f} N")
    print(f"Força média: {media:.2f} N")
    print(f"Somatório das reações: {soma_reacoes:.2f} N")

# =====
# FUNÇÃO: PLOTAGEM DA TRELIÇA ORIGINAL
# =====

def plot_trelica_original(N, x, y, barras, apoios):

    plt.figure(figsize=(10, 6))

```

```

# Barras
for barra in barras:

    i, j = barra

    plt.plot(
        [x[i], x[j]],
        [y[i], y[j]],
        'k-',
        linewidth=2
    )

# Nós
plt.scatter(x, y, color='blue', s=80)

# Numeração
for i in range(N):
    plt.text(x[i] + 0.05, y[i] + 0.05, f'{i}')

# Apoios
for i in range(N):

    if apoios[i] == "fixo":
        plt.plot(x[i], y[i] - 0.1, 'gs', markersize=12)

    elif apoios[i] == "movel":
        plt.plot(x[i], y[i] - 0.1, 'ro', markersize=10)

plt.title("Trelença Original")
plt.xlabel("X")
plt.ylabel("Y")
plt.grid(True)
plt.axis('equal')

# =====
# FUNÇÃO: TRELIÇA COLORIDA
# =====

def plot_trelenca_forcas(x, y, barras, forcas):

    plt.figure(figsize=(10, 6))

```

```

max_f = max(abs(f) for f in forcas)

for k, barra in enumerate(barras):

    i, j = barra

    intensidade = abs(forcas[k]) / max_f

    # Compressão -> vermelho
    # Tração -> azul

    if forcas[k] < 0:
        cor = (1, 1 - intensidade, 1 - intensidade)

    else:
        cor = (1 - intensidade, 1 - intensidade, 1)

    plt.plot(
        [x[i], x[j]],
        [y[i], y[j]],
        color=cor,
        linewidth=4
    )

plt.scatter(x, y, color='black')

plt.title("Mapa de Forças Internas")
plt.xlabel("X")
plt.ylabel("Y")
plt.grid(True)
plt.axis('equal')

# =====
# FUNÇÃO: GRÁFICO DE BARRAS
# =====

def plot_grafico_barras(forcas):

    plt.figure(figsize=(10, 6))

    indices = list(range(len(forcas)))

```

```

plt.bar(indices, forcas)

plt.xlabel("Barra")
plt.ylabel("Força Interna (N)")
plt.title("Magnitudo das Forças nas Barras")
plt.grid(True)

# =====
# FUNÇÃO: HISTOGRAMA
# =====

def plot_histograma(forcas):

    plt.figure(figsize=(10, 6))

    plt.hist(forcas, bins=10)

    plt.xlabel("Força (N)")
    plt.ylabel("Frequência")
    plt.title("Distribuição das Forças Internas")
    plt.grid(True)

# =====
# TRELIÇA-TESTE PADRONIZADA
# Warren simétrica
# =====

# Número de nós
N = 4

# Coordenadas
x = [0, 2, 4, 2]
y = [0, 0, 0, 2]

# Número de barras
B = 5

# Conectividade
barras = [
    [0, 1],
    [1, 2],

```



```

        [0, 3],
        [3, 2],
        [1, 3]
    ]

    # Cargas aplicadas
    Fx = [0, 0, 0, 0]
    Fy = [0, -1000, 0, 0]

    # Apoios
    apoios = [
        "fixo",
        "livre",
        "movel",
        "livre"
    ]

    # =====
    # EXECUÇÃO
    # =====

    inicio = time.time()

    # Montagem do sistema
    A, b, reacoes = montar_sistema(
        N,
        B,
        x,
        y,
        barras,
        Fx,
        Fy,
        apoios
    )

    # Solução
    solucao = gauss(A, b)

    fim = time.time()

    # =====
    # RESULTADOS
    # =====

```

```

forcas = solucao[:B]
reacoes_valores = solucao[B:]

tipos = classificar_barras(forcas)

print("\n===== RESULTADOS =====\n")

for i in range(B):

    print(f"Barra {i}:")
    print(f"Força = {forcas[i]:.2f} N")
    print(f"Tipo   = {tipos[i]}")
    print()

print("===== REAÇÕES =====")

for i, r in enumerate(reacoes):

    no, direcao = r

    print(
        f"Reação no nó {no} ({direcao}) = "
        f"{reacoes_valores[i]:.2f} N"
    )

print("\nTempo de execução:")
print(f"{fim - inicio:.6f} segundos")

# Estatísticas
estatisticas(forcas, reacoes_valores)

# =====
# GRÁFICOS
# =====

plot_trelica_original(N, x, y, barras, apoios)

plot_trelica_forcas(x, y, barras, forcas)

plot_grafico_barras(forcas)

plot_histograma(forcas)

```

```
plt.show()
```

===== RESULTADOS =====

Barra 0:

Força = 500.00 N

Tipo = TRAÇÃO

Barra 1:

Força = 500.00 N

Tipo = TRAÇÃO

Barra 2:

Força = -707.11 N

Tipo = COMPRESSÃO

Barra 3:

Força = -707.11 N

Tipo = COMPRESSÃO

Barra 4:

Força = 1000.00 N

Tipo = TRAÇÃO

===== REAÇÕES =====

Reação no nó 0 (x) = 0.00 N

Reação no nó 0 (y) = 500.00 N

Reação no nó 2 (y) = 500.00 N

Tempo de execução:

0.000218 segundos

===== ESTATÍSTICAS =====

Barra mais solicitada: Barra 4

Força máxima: 1000.00 N

Força média: 682.84 N

Somatório das reações: 1000.00 N



